The Confusion Matrix

Measuring a classifier: accuracy, precision, recall, F1

Marcus Birkenkrahe

COSC 3430 · Artificial Intelligence · Augustana University

Based on Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*, Ch. 3

Fall 2026

Where we are

- Pipeline step 6: select a model and train it. Step 7: fine-tune it – that needs a **performance measure**
- Regression (house prices): the RMSE
- Classification (kNN, decision trees): so far only "how many did we get right" – **accuracy**
- Today: why accuracy is not enough, and what to use instead

Question

A screening model is 95% accurate.

Is it a good model?

The scenario

A hospital deploys a model to screen **1,000 patients** for a rare disease.

- 50 patients actually have the disease
- 950 patients are healthy
- The model flags 80 patients as positive (disease detected)
- Of those 80 flagged, 40 truly have the disease

That is all you need. Everything else follows from these four numbers.

Four outcomes

Every patient is sick or healthy (*actual*); the model flags or clears them (*predicted*):

1. **True positive** (TP): flagged and actually sick
2. **False positive** (FP): flagged but healthy – a false alarm, *Type I error*
3. **True negative** (TN): healthy and correctly cleared
4. **False negative** (FN): sick but not flagged – a miss, *Type II error*

Which error is worse? It depends on the problem:

- Spam filter: FP = a real email lost in the spam folder $\Rightarrow$ FP costs more
- Cancer screening: FN = an undetected cancer $\Rightarrow$ FN costs more

Fill in the confusion matrix

→ **Switch to editor · code-along 1**

Reason it out from the scenario first, then check the sums.

	Predicted positive	**Predicted negative**
Actually positive	TP = 40	FN = 50 − 40 = 10
Actually negative	FP = 80 − 40 = 40	TN = 950 − 40 = 910

```
TP = 40   # flagged AND actually sick
FN = 10   # sick but not flagged
FP = 40   # flagged but actually healthy
TN = 910  # healthy and correctly cleared

print(f"TP + FN = {TP + FN} (all sick: 50)")
print(f"FP + TN = {FP + TN} (all healthy: 950)")
print(f"TP + FP = {TP + FP} (all flagged: 80)")
```

```
TP + FN = 50 (all sick: 50)
FP + TN = 950 (all healthy: 950)
TP + FP = 80 (all flagged: 80)
```

Accuracy

→ **Switch to editor · code-along 2**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Of all predictions, what fraction was correct? Compute it by hand first.

```
accuracy = (TP + TN) / (TP + TN + FP + FN)
print(f"Accuracy: {accuracy}")
```

```
Accuracy: 0.95
```

Question

A lazy model predicts “healthy” for every single patient.

What is its accuracy?

The lazy model

→ **Switch to editor · code-along 3**

It flags nobody: TP = 0, FP = 0, FN = 50, TN = 950.

```
TP0, FN0, FP0, TN0 = 0, 50, 0, 950
print(f"Lazy accuracy: {(TP0 + TN0) / 1000}")
print(f"Sick patients caught: {TP0} of {TP0 + FN0}")
```

```
Lazy accuracy: 0.95
Sick patients caught: 0 of 50
```

- Same 95% – and it catches nobody
- Accuracy fails on **imbalanced** (skewed) data: the 950 healthy patients dominate and hide complete failure on the 50 sick ones
- Géron's MNIST version: "not a 5" for every image is over 90% accurate

→ 95% accuracy says nothing until you know how balanced the classes are.

Precision

→ **Switch to editor · code-along 4**

$$\text{Precision} = \frac{TP}{TP + FP}$$

Of everyone the model *flagged*, what fraction truly had the disease?
How much do we trust a positive prediction?

```
precision = TP / (TP + FP)
print(f"Precision: {precision}")
```

```
Precision: 0.5
```

For every real case it catches, the model raises one false alarm.
Also called the *positive predictive value*.

Recall

→ **Switch to editor · code-along 5**

$$\text{Recall} = \frac{TP}{TP + FN}$$

Of everyone who *actually* had the disease, what fraction did the model catch?

```
recall = TP / (TP + FN)
print(f"Recall: {recall}")
```

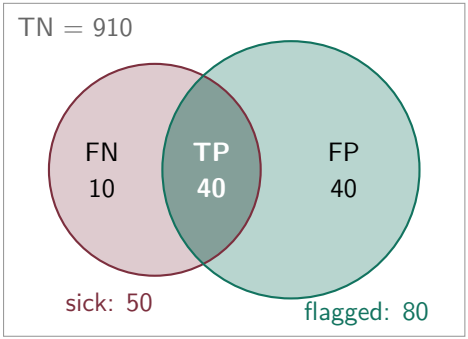
```
Recall: 0.8
```

Also called *sensitivity*, *true positive rate* (TPR), or the *power* of a test.

Look at the denominator: precision judges your flagged pool, recall judges what you missed.

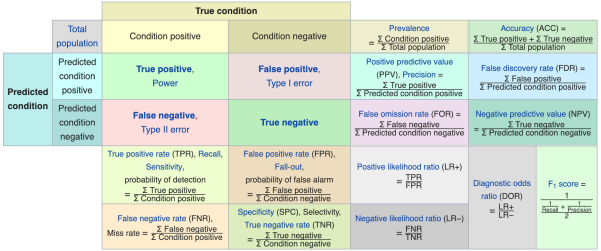
→ The lazy model: recall = 0/50 = 0, precision undefined.

Picture: two circles



- The overlap is TP = 40
- **Precision** = overlap / flagged circle = 40/80
- **Recall** = overlap / sick circle = 40/50
- Everything outside both circles: TN = 910

The full story



Source: rmartinshort.jimdofree.com/2019/02/10/intro-to-classification-metrics/

- **Specificity** = TN / (TN + FP) = 910/950 = 0.96 – recall for the negatives
- **Fall-out** (FPR) = FP / (FP + TN) = 40/950 = 0.04 – probability of a false alarm; you'll meet it in the ROC curve
- Careful: this chart has the *prediction* in the rows. scikit-learn puts the *actual* class in the rows.

The precision/recall trade-off

→ **Switch to editor · code-along 6**

A classifier computes a score per patient and flags everyone above a **decision threshold** (default 0.5).

Lower the threshold to 0.2: the model now flags **200** patients, 48 of them truly sick.

- Recall rises: FN drops from 10 to 2
- Precision falls: FP grows from 40 to 152

→ Lower threshold: higher recall, lower precision.

```
TP2 = 48
FN2 = 50 - TP2
FP2 = 200 - TP2
TN2 = 950 - FP2

precision2 = TP2 / (TP2 + FP2)
recall2 = TP2 / (TP2 + FN2)

print(f"t = 0.5: precision {precision:.2f}, "
      f"recall {recall:.2f}")
print(f"t = 0.2: precision {precision2:.2f}, "
      f"recall {recall2:.2f}")
```

```
t = 0.5: precision 0.50, recall 0.80
t = 0.2: precision 0.24, recall 0.96
```

confusion_matrix()

→ **Switch to editor · code-along 7**

```python
import numpy as np
from sklearn.metrics import (confusion_matrix, precision_score,
                             recall_score, f1_score,
                             classification_report)

y_true = np.array([0] * 950 + [1] * 50)   # healthy, sick
y_pred = np.array([0] * 910 + [1] * 40    # healthy: TN, FP
                  + [0] * 10 + [1] * 40)  # sick: FN, TP

print(confusion_matrix(y_true, y_pred))
print(f"Precision: {precision_score(y_true, y_pred):.2f}")
print(f"Recall:    {recall_score(y_true, y_pred):.2f}")
```

```
[[910  40]
 [ 10  40]]
Precision: 0.50
Recall:    0.80
```

Rows = actual, columns = predicted, label order (0 first):

	Pred. 0	**Pred. 1**
Actual 0	TN	FP
Actual 1	FN	TP

classification_report()

→ **Switch to editor · code-along 8**

```
print(classification_report(y_true, y_pred,
      target_names=["Healthy", "Disease"]))
```

```
              precision  recall  f1-score  support

     Healthy       0.99    0.96      0.97      950
     Disease       0.50    0.80      0.62       50

    accuracy                        0.95     1000
   macro avg       0.74    0.88      0.79     1000
weighted avg       0.96    0.95      0.96     1000
```

- **Per class**: that class treated as positive. Disease = our hand computation
- **support**: actual examples of that class
- **accuracy**: 0.95 – same as the lazy model
- **macro avg**: plain mean, $(0.99 + 0.50)/2 = 0.74$
- **weighted avg**: weighted by support, ≈ 0.96 – the big class hides the weak one

→ Watch the Disease row.

Question

Comparing models is easier with one number.

Why not just average precision and recall?

F1 and the harmonic mean

→ **Switch to editor · code-along 9**

The **F_1 score** is the *harmonic mean* of precision and recall:

$$F_1 = \frac{1}{\frac{1}{2}\left(\frac{1}{P} + \frac{1}{R}\right)} = \frac{2PR}{P+R}$$

Test it on an extreme model: it flags one patient, who is sick. $P = 1.0$, $R = 1/50 = 0.02$.

→ The arithmetic mean lets a perfect P hide $R = 2\%$. The harmonic mean is pulled toward the *lower* value.

```python
def f1(p, r):
    return 2 * p * r / (p + r)

print(f"Our model: F1 = {f1(precision, recall):.2f}")
print(f"sklearn:   F1 = {f1_score(y_true, y_pred):.2f}")

P, R = 1.0, 1 / 50
print(f"Extreme model: arithmetic mean "
      f"{(P + R) / 2:.2f}, F1 {f1(P, R):.2f}")
```

```
Our model: F1 = 0.62
sklearn:   F1 = 0.62
Extreme model: arithmetic mean 0.51, F1 0.04
```

Which models get a high F1?

→ Switch to editor · code-along 10

Guess first: which of the four gets a high F1?

```
cases = [("high P, high R", 0.90, 0.85),
         ("low P,  high R", 0.25, 0.90),
         ("high P, low R ", 0.90, 0.20),
         ("low P,  low R ", 0.25, 0.20)]
for label, p, r in cases:
    print(f"{label}: P {p:.2f}  R {r:.2f}  F1 {f1(p, r):.2f}")
```

```
high P, high R: P 0.90  R 0.85  F1 0.87
low P,  high R: P 0.25  R 0.90  F1 0.39
high P, low R : P 0.90  R 0.20  F1 0.33
low P,  low R : P 0.25  R 0.20  F1 0.22
```

→ Only high P *and* high R. One strong number cannot rescue a weak one – and that is not always what you want.

Practice: which metric would you choose?

Which error costs more – and do you optimize precision, recall, or F1?

1. **Cancer screening:** missing a case can be fatal. → Recall: FN can be fatal.
2. **Spam filter:** a real email in the spam folder may be lost. → Precision: FP loses a real email.
3. **Shoplifting camera:** every alert sends a guard to stop a customer. → Precision: FP accuses an innocent customer.
4. **Malware scanner:** undetected malware damages the system. → Recall: FN lets malware run.
5. **Follow-up test** with serious side effects for every flagged patient. → Precision: FP harms a healthy patient.
6. **Content moderation:** wrongly removed and missed harmful posts both do damage. → F1: balance both errors.

Géron reads the shoplifting case the other way: if alerts are cheap, high recall with low precision is fine. The answer depends on what an alert costs.

What we measured

Metric	Formula	Denominator	Answers
Accuracy	(TP+TN) / all	all predictions	How often are we right?
Precision	TP / (TP+FP)	predicted positive	When we flag, are we right?
Recall	TP / (TP+FN)	actual positive	How many real cases did we get?
F1	$2PR/(P+R)$	–	One balanced score

- On imbalanced data accuracy is almost useless: the lazy model scores 95%
- Moving the threshold trades precision against recall
- scikit-learn: `confusion_matrix()` (rows actual), `precision_score()`, `recall_score()`, `f1_score()`, `classification_report()`

Next: read Géron, Ch. 3, up to the end of "Performance Measures" (MNIST, precision/recall curve, ROC curve).